

Here's the extracted and rewritten **Java Coding Guidelines** from the image you provided, using the same professional and structured bullet-point style as before:

Java Coding Guidelines

Follow these guidelines to ensure code clarity, maintainability, and alignment with clean coding practices.

- Place similar or related methods together in your class.
For example, if two methods like `getFilteredList()` and `getTop5InFilteredList()` serve related purposes, define them consecutively to improve readability for future developers.
- Avoid IDE code warnings by handling conditions explicitly.
Example:
Instead of:

```
if (listIds == null && listIds.isEmpty())
```


Use:

```
if (listIds == null || listIds.isEmpty())
```
- Avoid placing business logic directly in controllers.
Move logic to a dedicated service class and call that service from the controller.
Example:

```
@GetMapping("/user")  
public User getUser() {  
    return userService.getUpdatedUser();  
}
```
- Avoid long and complex assertion chains in tests.
Break down repeated calls like `top5InFilteredListResponse.getLists().get(0).getItems()` into variables for better readability and debugging.
- Prefer Stream API over manual list processing with loops.
Example:
Replace:

```
List<String> result = new ArrayList<>();  
for(String s : list) {  
    if(s.startsWith("A")) {  
        result.add(s);  
    }  
}
```


With:

```
List<String> result = list.stream()  
    .filter(s -> s.startsWith("A"))  
    .collect(Collectors.toList());
```
- Avoid creating mocks and assertions directly in test classes.
Move mocks to a common utility class and reuse them to keep tests clean and focused.
- Avoid providing technical implementation details in Swagger API documentation.
Focus on meaningful business summaries and user-focused descriptions for clarity and API usability.
 - Avoid providing technical implementation details or internal summaries in Swagger API YAML files.
Swagger documentation might be read by non-technical users — keep it simple, user-oriented, and easy to understand.

- In Java test files, avoid hardcoding API URLs directly in tests.
Instead, define a constant for the URL and use that in test cases to simplify future updates and improve maintainability.
- Maintain proper formatting between comments and test data lines.
Add a space after `//` in section comments for clarity:

```
// Given
// When
// Then
```
- Move reusable test methods to a utility class inside the test folder.
Avoid code duplication and enhance test maintainability.
- Do not use magic numbers directly in code.
Replace them with named constants to improve readability and make the code self-explanatory.
- Avoid creating large methods with over 100 lines.
Break them down into small, private, reusable methods for better readability and testability.
- Use `VARCHAR2(n CHAR)` instead of `VARCHAR2(n)` in Oracle SQL queries.
The `CHAR` qualifier ensures the length is based on character count, not byte count, which is more reliable for multi-byte character sets.

Would you like a combined version of all Java guidelines compiled into one clean document?

From <<https://chatgpt.com/c/68137d16-6c18-8000-8385-30bc1f58085d>>